FARM TO TABLE

Local producers, sold direct. A marketplace with live auctions.

THE PROBLEM

THE SHELF PRICE IS NOT THE FARM PRICE

You pay at the supermarket	€1.00
Transport, warehouses, retail margin	~€0.80
What reaches the farmer	~€0.20

Farmers typically get 15–25% of the retail price (USDA ERS Food Dollar Series; similar EU figures).

Around **80%** of every euro stays in the middle.

We take 5%

IT IS NOT ONLY THE PRICE

DAYS ON A TRUCK. NO NAME ON THE BOX.

Freshness. Supermarket produce travels through warehouses for days. Here it goes from the farm to you.

Trust. Every product has a named producer, a place on the map and real reviews.

Surplus. A farmer with 200 kg of kiwi left over can auction it today instead of throwing it away.

QUEIJARIA DA BEIRA

AGED RESERVE CHEESE

Seia, Serra da Estrela · lot 147

THE DEAL

BOTH SIDES WIN

PRODUCERS

- Keep ~95% of the sale
- Set their own prices
- Sell surplus fast through auctions
- Get a public page, reviews and a sales dashboard

BUYERS

- Pay less than at the shelf
- Fresher food, from a known farm
- Follow every order step in real time
- Restaurants bid on bulk lots

The platform keeps a **5% fee** per sale. That is the business model.

FARM TO TABLE

Two ways to buy: a normal shop with a cart, and live auction rooms where everyone sees the same price and the same clock.

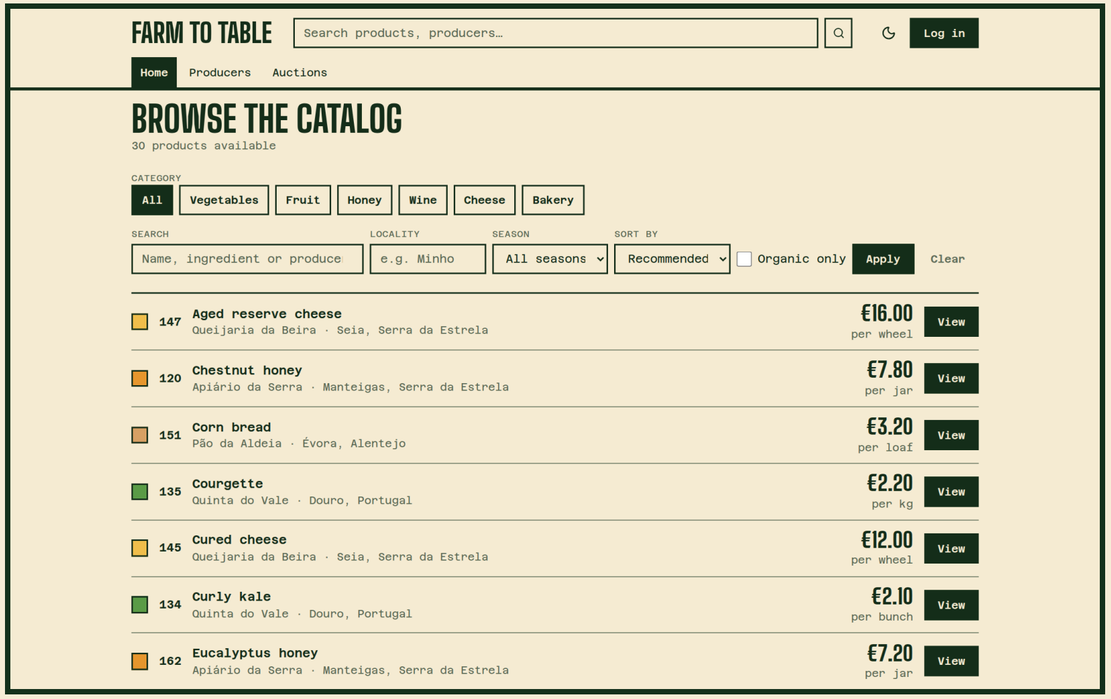
FEATURES, MOST IMPORTANT FIRST

WHAT IT DOES

01	LIVE AUCTIONS	Real-time bids over Socket.io, atomic on the server
02	SHOP, CART & ORDERS	Order status pushed to the buyer live
03	PRODUCER DASHBOARD	Products, stock, sales, orders to prepare
04	REVIEWS & MAP	Ratings after delivery, producer location
05	NOTIFICATIONS	Outbid, won, order moved
06	ADMIN BACKOFFICE	Approve producers, moderate reviews, status

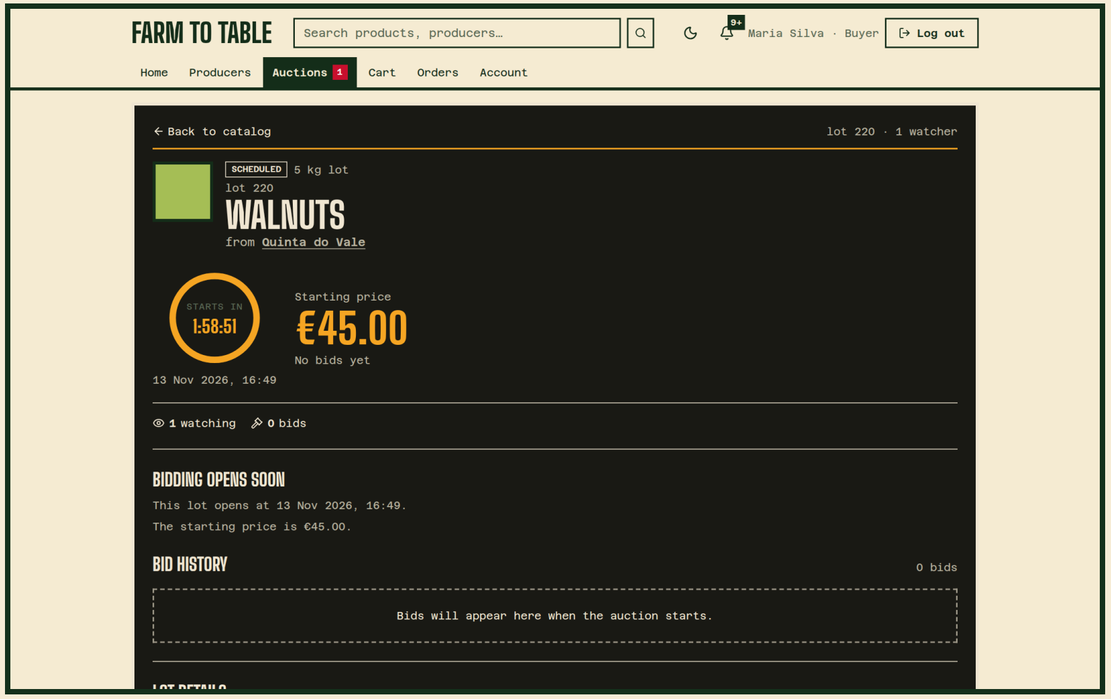
THE SHOP

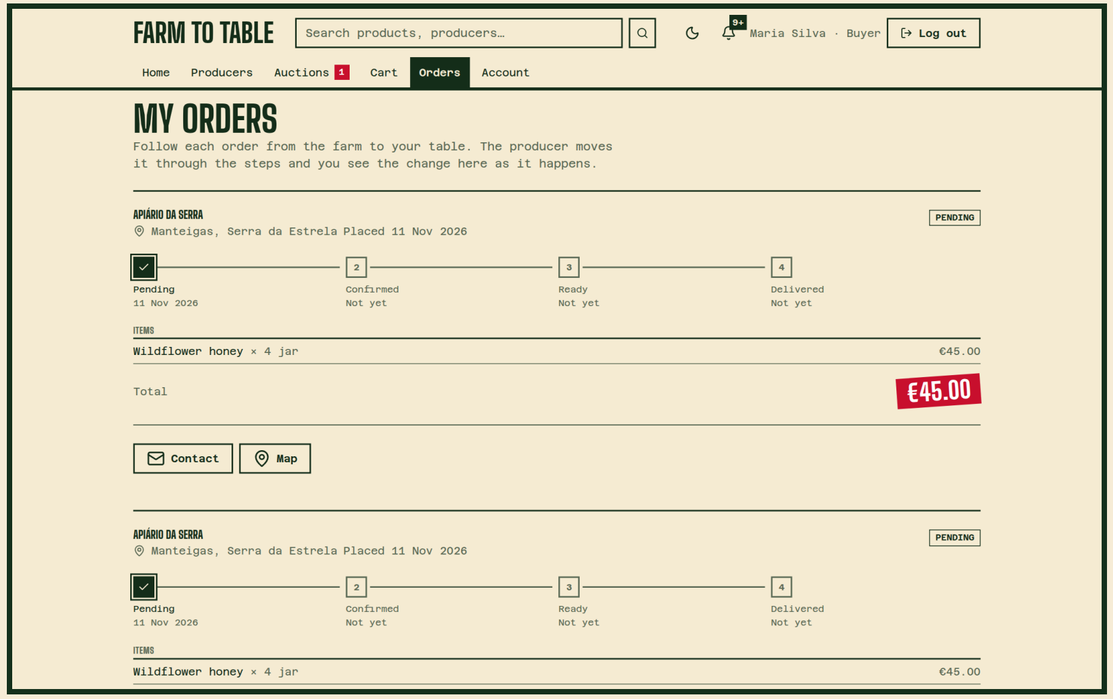
A MARKET PRICE BOARD





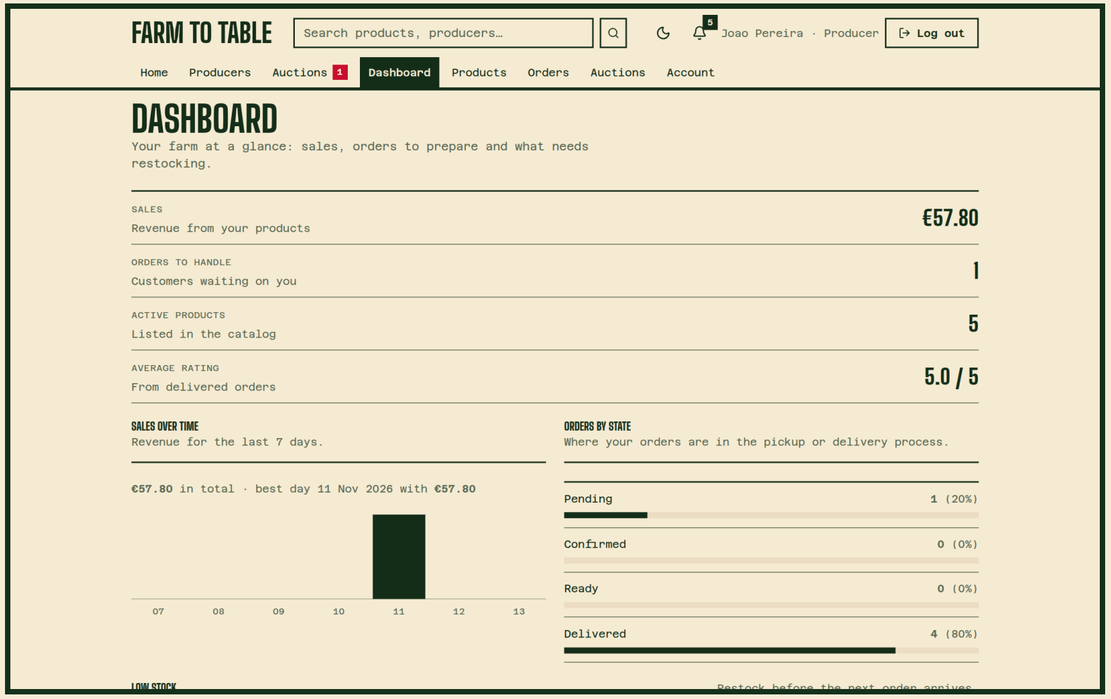
SAME PRICE, SAME CLOCK





FOR PRODUCERS & ADMINS

RUN THE FARM, RUN THE MARKET





TECH STACK

THREE APPS, ONE DATABASE

WEB

- React + Vite
- TypeScript
- Tailwind
- Socket.io client

API

- Node + Express
- Socket.io
- JWT auth
- OpenAPI docs

BACKOFFICE

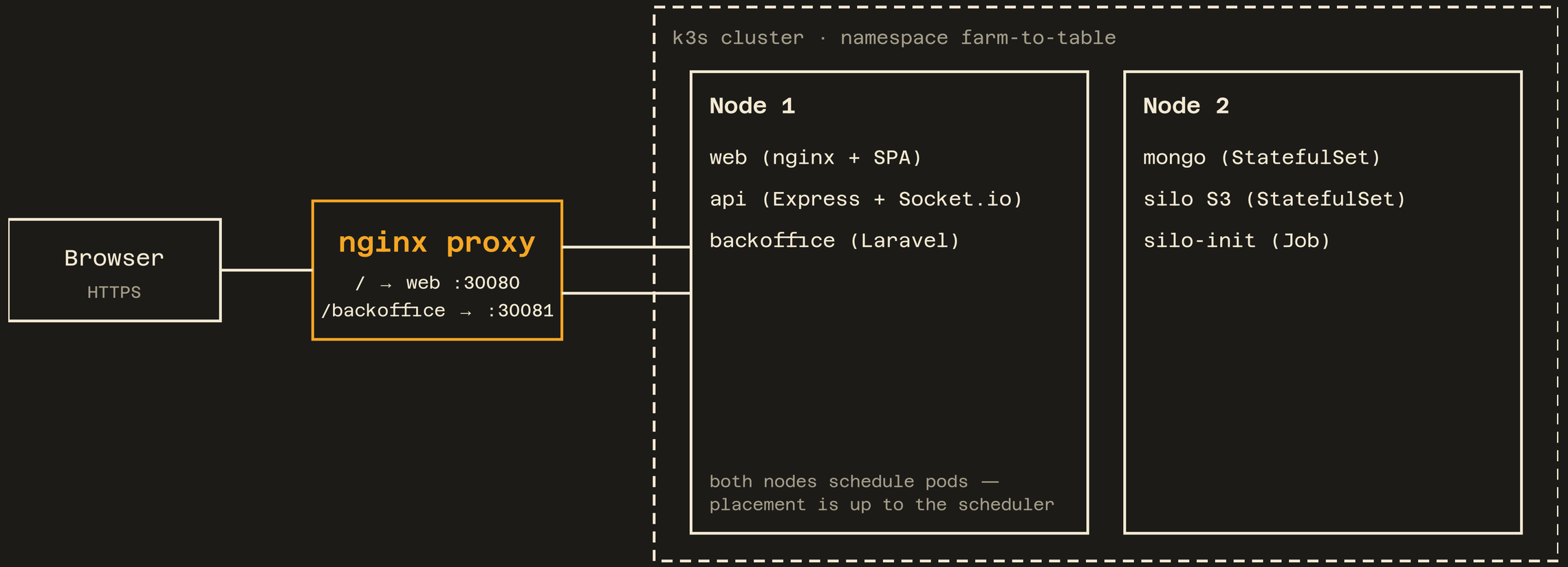
- Laravel 13
- Bootstrap 5
- Admin panel

DATA

- MongoDB 8
- S3 storage (silo) for images

PRODUCTION

HOW A REQUEST GETS IN



INFRA/K8S · ONE FILE PER SERVICE

INSIDE THE CLUSTER

NAMESPACE	`farm-to-table`, everything in one place
DEPLOYMENTS	api · web · backoffice
STATEFULSETS	mongo · silo, with persistent volumes
JOB	silo-init creates the image bucket
SERVICES	5 total; web & backoffice as NodePorts
SECRET	built from `.env` at deploy time

RULES WE KEPT

No secret in git: the Secret is generated on the node from .env

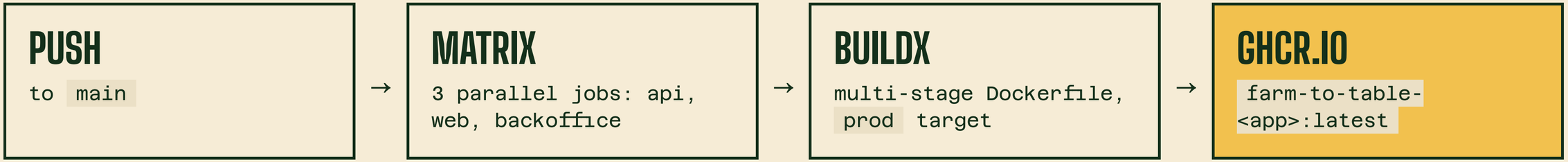
Public ports come from .env , not hard-coded

Images are public on ghcr.io, so the cluster needs no pull credentials

Same images run in compose.yaml locally and in k3s

.GITHUB/WORKFLOWS/IMAGES.YML

CI: GITHUB BUILDS THE IMAGES



GitHub only builds. It never touches the servers, so no server credentials live in GitHub.

INFRA/JENKINS/JENKINSFILE

CD: JENKINS DEPLOYS

POLL
checks GitHub every 2 min

→

WAIT
until the image build for this exact commit succeeds

→

PULL
SSH to the node, `git pull`, check `.env`

→

APPLY
Secret, manifests, `rollout restart`

→

CHECK
`rollout status` for each app

Polling means GitHub never needs to reach Jenkins: **no public webhook**.

On failure it prints the pod states and the last cluster events, so the log says **why**.

COMMIT TO PRODUCTION

ONE GIT PUSH

01	GIT PUSH MAIN	developer
02	3 IMAGES BUILT & PUSHED	GitHub Actions → ghcr.io
03	JENKINS SEES THE COMMIT	pollSCM, waits for images
04	KUBECTL APPLY + RESTART	k3s, 2 nodes
05	LIVE	farm-to-table.maruqes.com

DEMO